



南開大學  
Nankai University

南 開 大 學

計 算 機 學 院

並行程序设计实验报告

---

## 协程技术调研

---

姓名：吕昊远

学号：2410668

专业：计算机科学与技术

指导教师：王刚

2026 年 6 月 4 日

目录

一、 引言 1

二、 并发模型的演进与协程的提出 1

    (一) 进程到协程的历史演进 1

    (二) 线程模型在高并发 I/O 场景下的开销 1

三、 协程的本质与底层机制 2

    (一) 协程的本质 2

    (二) 实现方式 2

    (三) 无栈协程 2

    (四) 有栈协程 2

    (五) 协作式调度与事件循环 3

四、 三方协同与主流语言支持 3

    (一) 编译器、运行时与操作系统的协同 3

    (二) 主流语言的协程支持 3

五、 性能对比实验 4

    (一) 实验设计与方法 4

    (二) 结果分析 6

六、 适用边界与工程选型 7

七、 总结与展望 7

八、 附录: 代码仓库链接 7

## 一、 引言

当我们随手打开一个应用或是点开一个网页时, 这些在客户端看似平常的操作, 对其背后的服务器而言, 往往意味着同时处理成千上万个来自用户的请求。如今我们可以较为从容地应对这类高并发场景, 是因为业界已经积累了一套成熟的方法; 然而在十几年前, 这其实并非易事。一台服务器很容易被大量并发连接压垮。本报告所讨论的协程 (Coroutine) 技术, 正是为了更优雅地解决这一问题而发展起来的, 或者更确切地说, 是一项被工程问题推动而重新拾起的技术。

从机制上看, 协程所做的, 是把“应该让谁运行”这一调度决定, 从操作系统内核转移到用户态, 用一个轻量的状态机或一段很小的栈, 来代替开销昂贵的内核线程。正因如此, 在高并发、且执行时间主要消耗在 I/O 等待的场景中, 协程能够以更少的资源换取数量级的扩展能力, 这也使其成为现代互联网工程的核心技术之一。

本次调研报告主要分为三部分: 其一是协程的**机制**, 即协程如何挂起与恢复, 以及有栈与无栈两种实现路线; 其二是计算机内部如何进行多方的**协同**, 即编译器、运行时与操作系统三者如何分工来实现协程; 其三是用于验证的**实验测试**, 在本次调研中还设计了几个的实验, 从内存占用、创建开销、I/O 吞吐与上下文切换延迟四个方面, 对比协程与线程的差异与特征。

## 二、 并发模型的演进与协程的提出

### (一) 进程到协程的历史演进

站在当下的视角回看, 并发模型经历了从进程、到线程、到线程池、再到协程的演进过程。这一过程始终被同一个问题推动着, 即如何让一台机器同时处理更多的任务。

最原始的做法是“一个任务一个进程”, 但是进程过于笨重, 其创建与切换的开销都很大; 于是出现了线程, 线程比进程更加轻量, 但其本质仍由操作系统内核调度; 当任务数量增多时, 频繁创建与销毁线程又成为负担, 因而进一步发展出线程池, 将线程提前建好并反复使用。然而, 随着互联网规模的扩张, 当并发连接达到上万乃至数十万的量级时, 即便是线程池也难以支撑。这也是历史上著名的 C10K 问题, 即如何让一台机器同时处理一万个连接。

值得指出的是, 协程并非全新的发明。这一概念早在上世纪五六十年代就已被提出, 只是在很长一段时间里未被主流采用。从调度方式上看, 我们今天熟知的线程对应“抢占式多任务”, 而协程对应“协作式多任务”。在操作系统发展的历史岔路口上, 出于对安全性与公平性的考量, 人们最终优先发展了抢占式的线程; 协程则更像是一项被现实问题重新唤起的旧思想, 在今天的高并发场景下重新进入了我们的视野。

### (二) 线程模型在高并发 I/O 场景下的开销

要理解协程的价值, 首先要理解线程为什么有高昂开销。其开销主要来自三个方面。

- **栈内存:** 对于每个线程来说, 默认都需要预留一段虚拟栈, 常见大小在 8 MB 量级。在万级并发下, 仅此一项就会对地址空间与实际驻留内存 (RSS) 构成巨大的压力。
- **切换成本:** 在线程切换时, 必须要陷入内核、重新调度, 并伴随 TLB 与 CPU 缓存失效, 其延迟通常可达微秒量级。
- **创建成本:** 创建一个线程也需要经由系统调用, 在内核中建立相应的对象, 在后面的实验测试中, 每个线程的创建耗时约 67  $\mu$ s。

从中我们也能看出,问题的根源在于线程把每一条并发流的创建、切换与内存开销都放在了内核之中。每一条并发流都是内核对象,其创建、切换与内存管理都需要内核参与。协程的核心思路,正是把这部分开销从内核转移到用户态。大量的协程可以复用同一个线程,它们之间的切换开销与一次普通函数调用相近。这就是协程更为轻量的根本原因。

## 三、 协程的本质与底层机制

### (一) 协程的本质

在明确了线程的局限之后,可以从功能的角度对协程给出一个较为准确的描述: **协程是一种可以在执行中途主动暂停、交出控制权,之后又能从原来的位置精确恢复的函数。**

普通函数一经调用,就必须运行至 `return` 才会交回控制权,中途无法停下来转而去别的事情。而协程可以在 `await` 或 `yield` 等位置主动“挂起”,将线程让给其它的执行流,等待条件就绪后再被“恢复”。其关键在于当被挂起时,协程的局部变量与执行位置会被完整保存下来。

考虑同一线程上的时间线:协程 A 运行过程中遇到一次 I/O 等待,它并不像普通函数那样原地阻塞等待,而是直接挂起,把线程让给协程 B;待 A 所等待的 I/O 就绪后,再回到此前的断点继续执行。这样,一个线程便可以多个协程高效地轮流使用。通过这个具体的流程,我们可以归纳出协程的三个特征: **轻量、协作式、以及可以用同步的写法表达异步逻辑。**

### (二) 实现方式

在具体实现上,协程当前主要分为两大流派,二者的底层机制有所差异。

**无栈协程 (Stackless)** 的代表是 C++20、Python、C#。它没有独立的运行栈,而是把状态保存在堆上的一个“协程帧”中,由编译器把函数改写为状态机来实现。其优点是节省内存、切换极快;缺点则是写法较为受限,需要 `async/await` 这类关键字,并由此带来“函数染色”等问题。

**有栈协程 (Stackful)** 的代表是 Go 的 `goroutine` 与 Java 21 的虚拟线程。它为每个协程配备一个很小、但可按需增长的独立栈,运行时通过切换栈指针 (SP) 来完成挂起与恢复。其优点是直接写普通的代码即可,无需特殊的关键字,且可在调用栈的任意深度进行挂起;代价则是每个协程会多占用一些内存。

二者的核心差异可以概括为:无栈协程更省内存、切换更快,但写法受限;有栈协程写法自然、可在任意深度挂起,代价是每个协程会占用略多内存。二者并无绝对优劣,而是不同的设计取向,和语言自身的风格以及开发理念相关。

### (三) 无栈协程

对于一段含有两个挂起点的异步函数。编译器在编译期间会将其“切分”为一台状态机:入口为状态 0,执行到第一个 `await` 即挂起;下次进入时为状态 1,从上次的位置继续;依此类推,直至完成后销毁。挂起时,原本位于栈上的局部变量会被搬移到堆上的“协程帧”中,与“当前状态号”一并保存。因此,恢复的本质就是跳转到对应状态继续执行。这也解释了为什么无栈协程不需要独立的栈,且其切换非常轻量,开销仅仅约等于一次函数跳转。

### (四) 有栈协程

有栈协程的思路则朴素得多:有  $N$  个协程,就有  $N$  个独立的小栈,每个栈初始化时可能仅有约 2 KB,但是会动态地按需增减。挂起与恢复的本质,是运行时在用户态保存当前协程的栈指针与寄存器,再载入下一个协程的相应状态,整个过程也不会涉及内核态的操作。

这一设计带来三点好处: 其一, 由于整条调用栈都被保留, 因此可以在函数嵌套很深的地方挂起; 其二, 写法没有侵入性, 普通函数即可, 也无需特殊的关键字; 其三, 为协程分配的栈既能增长也能收缩, 在节省内存的同时不易发生栈溢出等问题。

综合对比来看, 无栈协程把状态压缩进一个堆帧, 切换约等于一次跳转; 有栈协程则保留一整条完整的栈, 可在任意深度挂起。前者更为高效, 但写法受限; 后者则更加自然, 但是每个协程占用略高。

## (五) 协作式调度与事件循环

对于协程来说, 仅能挂起还远远不够, 还需要有机制来决定接下来该恢复哪个协程, 以及协程所等待的 I/O 什么时候就绪。这正是调度器与事件循环承担的核心职责。一次完整的 I/O 流程如下:

- (1) 协程执行到 `await`, 向操作系统注册其关注的文件描述符, 随后挂起;
- (2) 事件循环直接转去运行其它就绪的协程;
- (3) 操作系统通过 `epoll` / `io_uring` 等机制, 在某文件描述符就绪时通知事件循环;
- (4) 事件循环将对应的协程放回就绪队列, 并恢复其执行。

这里也体现了协程区别于线程的最本质差异, 即协作式调度没有抢占。也就是说, 如果某个协程执行了一段阻塞调用, 或长时间进行纯计算而不主动挂起, 那么整个事件循环都会被它阻塞。这是协程编程中最常见的隐患。实际上, 从这一点我们也可以理解, 为什么操作系统层面当年没有优先采用协作式调度。在协作式模型下, 只要有一处任务卡死, 整个系统都可能面临崩溃的问题, 而相比之下, 抢占式的线程明显在安全性上更有保障。即便在今天的工程实践中, 协程也需要谨慎地使用, 或由运行时为其划定明确的边界。

## 四、三方协同与主流语言支持

### (一) 编译器、运行时与操作系统的协同

将前面的这些机制串联起来可以看到, 协程并非某一层独立的实现, 而是编译器、运行时与操作系统三方协同的结果, 三者都有自己的功能:

- **编译器 / 语言:** 负责“如何暂停”, 把 `async` 函数改写为状态机, 确定挂起点与局部变量的保存布局, 并提供 `await` / `co_await` 等原语;
- **运行时 / 调度器:** 负责“先运行谁”, 通过事件循环与就绪队列决定恢复的顺序, 以  $M:N$  的方式将  $M$  个协程复用到  $N$  个操作系统线程之上, 并封装异步 I/O 接口;
- **操作系统内核:** 负责“何时唤醒”, 提供 `epoll` / `io_uring` / `IOCP` 等就绪通知机制, 真正执行非阻塞 I/O, 并把承载线程调度到 CPU 核心上。

### (二) 主流语言的协程支持

在实际开发场景中, 不同语言对协程的支持体现出各自的设计取向, 这里我们做一个简单的总结。

表 1: 五种主流语言的协程支持对比

| 语言/版本             | 协程类型 | 调度方式       | 关键原语                                           | 栈/帧           | 运行时          |
|-------------------|------|------------|------------------------------------------------|---------------|--------------|
| C++20             | 无栈   | 由库决定       | <code>co_await</code><br><code>co_yield</code> | / 堆上协程帧       | 需三方库 (asio)  |
| Go                | 有栈   | 运行时 M:N    | <code>go + channel</code>                      | 可增长小栈<br>≈2KB | 内建 GMP       |
| Python<br>asyncio | 无栈   | 单线程事件循环    | <code>async / await</code>                     | 协程对象 (堆)      | asyncio 事件循环 |
| C#(.NET)          | 无栈   | 线程池 + 任务调度 | <code>async/await Task</code>                  | + 状态机 (堆)     | TPL / 线程池    |
| Java 21           | 有栈   | 挂载到载体线程    | 虚拟线程 (普通代码)                                    | 可暂存连续栈        | Loom / JVM   |

从中可以看出, 越是追求底层控制的语言 (如 C++), 越注重于高效性, 把选择权交给程序员; 而越是以并发为优先的语言, 则越倾向于把调度器内置到运行时, 让使用更加方便。

## 五、性能对比实验

### (一) 实验设计与方法

为了验证协程的这些特性, 本次调研中还设计并完成了一组实验。实验采用单核服务器, 涉及的语言有 Python 3.12、Go 1.22、g++ 13.3(C++20)。实验主要从以下四个方面展开:

表 2: 实验设计

| 实验     | 测量方法                                                   | 测量指标             |
|--------|--------------------------------------------------------|------------------|
| 内存占用   | 创建 $N$ 个空闲并发单元并全部挂起, 测量进程 RSS 净增量                      | KB/单元            |
| 创建开销   | 批量创建并完成大量空任务, 测量平均创建耗时                                 | $\mu\text{s}$ /个 |
| I/O 吞吐 | 2000 个请求, 每个以 <code>sleep 20ms</code> 模拟 I/O, 测量总耗时与吞吐 | req/s            |
| 切换延迟   | 紧凑循环内反复挂起/恢复, 或双线程相互唤醒                                 | ns/次             |

在单核环境下, 不存在多核并行带来的其他影响, 从而让我们更加便于聚焦协程在“减少等

待”上的本质。接下来，我们对上述实验进行深入的详细分析。

## 2. 内存占用与创建开销

内存占用与创建开销的实测结果分别如下：

表 3: 每并发单元的内存占用与创建开销

| 并发单元      | 内存占用 (KB/单元) | 创建耗时 ( $\mu\text{s}$ /个) |
|-----------|--------------|--------------------------|
| OS 线程     | 16           | 67                       |
| Go 协程     | 2.6          | 1.15                     |
| Python 协程 | 1.1          | 11.3                     |

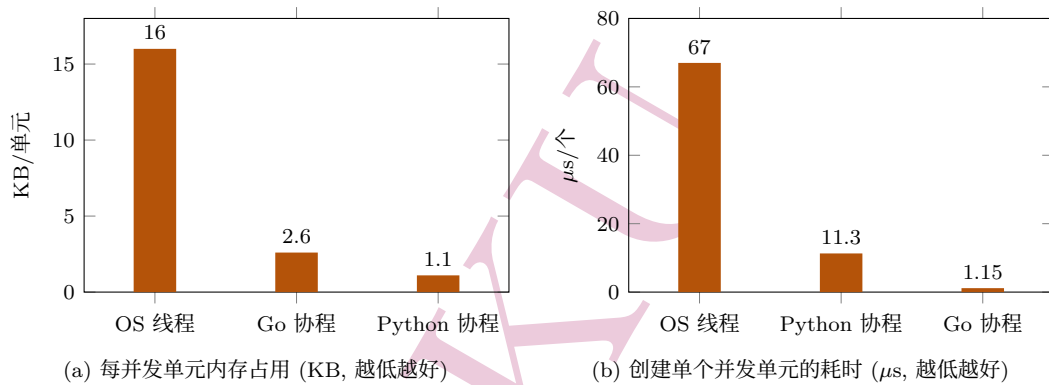


图 1: 内存占用与创建开销对比

实验数据表明, 线程约占 16 KB/单元, 而协程约为 1~3 KB, 内存占用相差 5~15 倍; 在创建开销上, OS 线程为 67  $\mu\text{s}$ , 而 Go 协程仅 1.15  $\mu\text{s}$ , 相差可达近 60 倍。在相同的内存下, 协程能够支撑的并发量可以比线程高出一个数量级, 具有显著的效率优势。因此我们可以说, “百万级协程”是现实可行的, 而 “百万级线程” 则几乎是不可能的。

### 1. I/O 吞吐与上下文切换延迟

I/O 吞吐的实测结果如下。其中每个请求以 `sleep 20 ms` 模拟一次简单可控的 I/O 等待, 以隔离并发模型自身的差异。

表 4: 处理 2000 个 I/O 请求的总耗时与吞吐

| 方案               | 总耗时 (ms) | 吞吐 (req/s) |
|------------------|----------|------------|
| 线程池 (50)         | 812      | 2464       |
| 线程池 (200)        | 232      | 8620       |
| 线程池 (1000)       | 225      | 8902       |
| 协程 (单线程 asyncio) | 37.5     | 53352      |



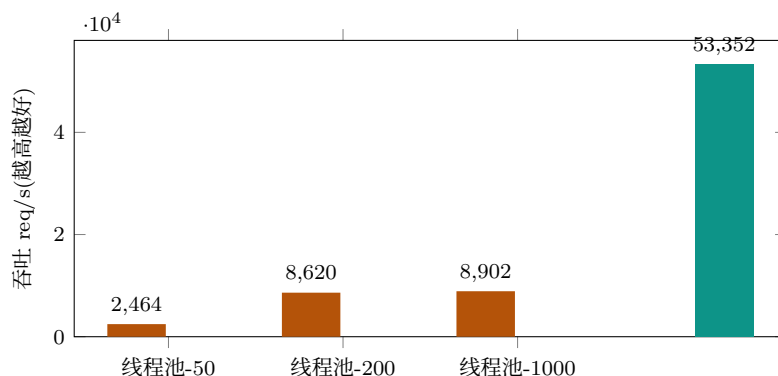


图 2: I/O 吞吐对比 (2000 个请求, 每个等待 20ms)

从中我们可以清晰地看出, 线程池在 50 个工作线程时吞吐约为 2464 req/s; 扩容到 200 甚至 1000 时, 吞吐几乎不再增长 (8620 与 8902 req/s)。这是因为线程数过多时, 调度与竞争的开销会抵消掉绝大部分的增益。而相比之下, 单线程协程的吞吐则高达 53352 req/s, 约为 50 线程池的 21 倍。

上下文切换延迟的实测结果如表 5 所示。

表 5: 上下文切换延迟对比

| 切换类型                        | 延迟 (ns/次) | 相对倍数   |
|-----------------------------|-----------|--------|
| OS 线程切换 (condvar)           | 2045      | ≈1100× |
| C++20 协程切换 (resume/suspend) | 1.85      | 1×     |

C++20 协程一次挂起与恢复仅需约 1.85 ns, 而 OS 线程切换需约 2045 ns, 二者相差约 1100 倍。前者约等于一次普通函数调用, 后者则必须陷入内核。这也正是协程在 I/O 密集场景下取得显著优势的微观原因。

## (二) 结果分析

综合上述实验, 分析协程在 I/O 场景占优的原因, 主要有三点。首先是等待不占用线程, 大量 I/O 等待可以重叠在同一个线程上完成; 其次则是协程的切换均在用户态完成, 无需系统与内核调度, 且对缓存友好; 同时, 协程还省去了线程竞争、上下文切换与栈内存的全部开销。

然而, 协程也存在其局限与适用边界:

- **不能加速 CPU 密集任务:** 协程不提供并行运算的能力, 纯计算仍需依赖多核或多进程;
- **没有抢占:** 单个阻塞调用或死循环即可阻塞整个事件循环;
- **函数染色:** 无栈协程中, async 会沿调用链向上传染;
- **依赖生态配合:** 如果引入同步阻塞库, 异步带来的优势便会消失。

由此可得出本次调研的核心结论: **协程并不是“更快的线程”, 它消除的是等待的开销, 而非提供额外的算力。** 它将 I/O 等待的成本压缩至接近于零; 但当需要真正的并行算力, 仍须回到多核与多进程。将计算密集部分交由线程池或进程池, 让协程专注处理 I/O, 是较为稳妥的混合方式。



## 六、 适用边界与工程选型

基于上述分析, 在实际工程中可以按照负载特征, 将选型大体分为三类。

- **I/O 密集 + 高并发** (如 Web/API 服务、网关、数据库与 RPC 代理、海量长连接):  
**优先采用协程;**
- **CPU 密集 / 重计算** (如大量数值运算):  
**采用并行而非协程**, 例如多进程、多线程、SIMD 或 GPU 等, 协程在此场景下没有收益;
- **混合负载:**  
**采用混合方案**, 以协程的主循环处理 I/O, 把计算密集部分交给进程池或线程池, 注意避免在协程内编写阻塞调用。

在判断选择技术实现时, 首先要分析瓶颈究竟在“等待”还是“计算”: 等待则采用协程, 计算则采用并行; 同时需要确认整条调用链是否均为异步, 因为只要混入一个同步阻塞库, 协程的异步优势便会被抵消。

## 七、 总结与展望

本次调研中, 我们对协程技术进行了较为系统的梳理, 从底层到应用, 全面调研了协程这一项并发技术。

在本质上, 协程是用户态、协作式的可挂起函数, 将调度由内核转移至用户态, 是其各项优势的根源。在机制上, 无栈协程以编译器生成的状态机加堆帧实现, 有栈协程以运行时管理的小栈加栈指针切换实现, 二者背后都是语言、运行时与操作系统的三方协同。其中, 我们还进行了实验测试, 协程相对线程的内存占用相差 5~15 倍, 创建开销相差数十倍, 上下文切换延迟相差约 1100 倍, I/O 吞吐约高 21 倍; 但是对于 CPU 密集任务, 协程则无法带来任何收益。

展望未来, 这一领域仍在快速发展之中。C++26 正在标准化统一的异步执行模型, Python 也正逐步移除 GIL。我们可以预见, 协程与多核并行的混合分工, 将成为工程上的重要技术手段, 协程的使用也将变得更加高效、更加方便。

## 八、 附录: 代码仓库链接

**GitHub:** <https://github.com/Iriasoul/Parallel-Programming/tree/main/labX-coroutine>